# Photopic

A Luminous Efficacy Optimizer for Spectral Power Distributions

User Manual

March 2026

## Contents


**1 Introduction** **3**
- 1.1 Background . . . 3
- 1.2 How it works . . . 3

**2 Mathematical Foundations** **3**
- 2.1 Planck's law and the blackbody reference . . . 3
- 2.2 Luminous efficacy of radiation . . . 4
- 2.3 Tristimulus values and chromaticity . . . 4
- 2.4 Correlated color temperature . . . 4
- 2.5 Color Rendering Index . . . 4
- 2.6 The optimization problem . . . 5
- 2.7 Philips phosphor-converted LED model . . . 6
- 2.8 Parametric spectrum . . . 6

**3 Building** **7**
- 3.1 Prerequisites . . . 7
- 3.2 Build procedure . . . 7

**4 Command-Line Interface** **8**
- 4.1 `-run`: Maximize efficacy . . . 8
- 4.2 `-run-maxr9`: Maximize $R_9$ (inverse problem) . . . 8
- 4.3 `-run-maxr9-philips`: Maximize $R_9$ with Philips correction . . . 9
- 4.4 `-run-drop`: Multi-run with TCS axis dropping . . . 9

**5 Worked Examples** **9**
- 5.1 Example 1: Basic run at CRI $\geq 80$ . . . 9
- 5.2 Example 2: High CRI at CRI $\geq 95$ . . . 9
- 5.3 Example 3: Moderate CRI with $R_9$ constraint . . . 10
- 5.4 Example 4: Inverse problem—maximize $R_9$ at efficacy $\geq 350$ . . . 10
- 5.5 Example 5: Running the full CRI sweep . . . 10
- 5.6 Example 6: Sweeping the efficacy–$R_9$ frontier . . . 10
- 5.7 Example 7: Single Philips-corrected run . . . 10
- 5.8 Example 8: Plotting results . . . 10

**6 Output Files** **11**
6.1 File naming convention . . . 11
6.2 `.res` files: Optimization results . . . 11
6.3 `.dat` files: Spectral data . . . 12

**7 Architecture** **12**
7.1 Parametric spectrum model . . . 12
7.2 Hierarchical subdivision . . . 12
7.3 COBYLA optimizer . . . 13
7.4 CRI calculation . . . 13
7.5 Constraint convention . . . 14

**8 Interactive Use** **14**
8.1 Useful interactive commands . . . 14

**9 Customization** **15**
9.1 Writing new objective functions . . . 15
9.2 What `calc-specs` returns . . . 16
9.3 Accessing individual fields . . . 16
9.4 Example: maximize efficacy subject to $R_9 \geq 50$ . . . 16

**10 Reference** **17**
10.1 Global variables . . . 17
10.2 Key source files . . . 17
10.3 Physical constants . . . 17

# 1 Introduction

`photopic` is a constrained optimization tool that finds spectral power distributions (SPDs) maximizing the *luminous efficacy of radiation* (LER) or the $R_9$ color rendering index, subject to constraints on CRI (Color Rendering Index), CCT (Correlated Color Temperature), $D_{uv}$ (distance from the Planckian locus), and individual color rendering components $R_i$.

The tool answers questions of the form: "What is the theoretical maximum luminous efficacy achievable at 2700 K with CRI $\geq$ 80?" and, conversely, "What is the best $R_9$ (saturated red rendering) achievable at 2700 K with CRI $\geq$ 80 and efficacy $\geq$ 350 lm/W?"

## 1.1 Background

The luminous efficacy of radiation (LER) measures how efficiently a spectrum produces visible light, in lumens per watt of radiated power. The theoretical maximum is 683 lm/W, achieved by monochromatic light at 555 nm—which has a CRI of zero. Real light sources must balance efficacy against color quality: broader spectra render colors more faithfully but waste energy in wavelength regions where the eye is less sensitive.

`photopic` explores this trade-off computationally, using the COBYLA derivative-free constrained optimizer to search a parametric space of spectral shapes. Color rendering is evaluated via the CIE 13.3 test color method with 14 test color samples (TCS).

## 1.2 How it works

`photopic` starts from a blackbody spectrum at the target CCT and applies a multiplicative modifier of the form $\exp(f(\lambda))$, where $f(\lambda)$ is a piecewise-linear function defined by a parameter vector. The optimizer adjusts these parameters to minimize (or maximize) the objective while satisfying the constraints.

To avoid poor local minima in the high-dimensional space, the optimization uses *hierarchical subdivision*: it begins with 2 parameters (a single linear slope across the visible spectrum), optimizes, then doubles the resolution to 3, 5, 9, 17, 33, 65, and finally 129 parameters (approximately 3 nm spacing across 380–770 nm). At each level, the COBYLA step size is halved.

# 2 Mathematical Foundations

This section summarizes the physics and colorimetry underlying the optimization. For a full treatment and computational results, see the companion report [1].

## 2.1 Planck's law and the blackbody reference

The spectral radiance of a blackbody at temperature $T$ is

$$B(\lambda, T) = \frac{2hc^2}{\lambda^5} \cdot \frac{1}{\exp\big(hc/\lambda kT\big) - 1} \tag{1}$$

where $h$ is Planck's constant, $c$ the speed of light, and $k$ Boltzmann's constant. This serves as the starting point for the optimization: the parametric spectrum is a multiplicative perturbation of $B(\lambda, T)$.

## 2.2 Luminous efficacy of radiation

The luminous efficacy of radiation (LER) measures how effectively a spectral power distribution $S(\lambda)$ produces the sensation of brightness:

$$\eta = K_m \frac{\displaystyle\int_{\lambda_0}^{\lambda_1} S(\lambda)\, V(\lambda)\, d\lambda}{\displaystyle\int_{\lambda_0}^{\lambda_1} S(\lambda)\, d\lambda} \tag{2}$$

where $V(\lambda)$ is the CIE photopic luminous efficiency function (peaking at unity at 555 nm), $K_m = 683\,\mathrm{lm/W}$ is the maximum luminous efficacy, and $[\lambda_0, \lambda_1] = [380, 770]$ nm spans the visible spectrum. For monochromatic light at 555 nm, $\eta = 683\,\mathrm{lm/W}$; for a broadband source, $\eta$ is necessarily lower. *This is the primary quantity that* `photopic` *maximizes.*

## 2.3 Tristimulus values and chromaticity

The CIE 1931 tristimulus values of a spectrum $S(\lambda)$ are

$$X = \int_{\lambda_0}^{\lambda_1} S(\lambda)\, \bar{x}(\lambda)\, d\lambda, \quad Y = \int_{\lambda_0}^{\lambda_1} S(\lambda)\, \bar{y}(\lambda)\, d\lambda, \quad Z = \int_{\lambda_0}^{\lambda_1} S(\lambda)\, \bar{z}(\lambda)\, d\lambda \tag{3}$$

where $\bar{x}$, $\bar{y}$, $\bar{z}$ are the CIE 1931 2° standard observer color matching functions, tabulated at 1 nm resolution in `CieXyz.i3`. The chromaticity coordinates are

$$x = \frac{X}{X+Y+Z}, \qquad y = \frac{Y}{X+Y+Z} \tag{4}$$

and the CIE 1960 UCS coordinates used for CCT determination are

$$u = \frac{4X}{X+15Y+3Z}, \qquad v = \frac{6Y}{X+15Y+3Z}. \tag{5}$$

## 2.4 Correlated color temperature

The correlated color temperature (CCT) of a source is the temperature $T$ of the Planckian radiator whose chromaticity is nearest in $(u, v)$ space:

$$\mathrm{CCT} = \arg\min_T \left\| (u_{\mathrm{source}}, v_{\mathrm{source}}) - (u_{\mathrm{Planck}}(T), v_{\mathrm{Planck}}(T)) \right\| \tag{6}$$

The signed distance from the Planckian locus is $D_{uv}$, constrained to $D_{uv} \leq 0.012$ to ensure the source chromaticity lies acceptably close to the blackbody locus. `photopic` determines the CCT by Brent's method search over a precomputed temperature-to-$uv$ lookup table (`TempUv.m3`).

## 2.5 Color Rendering Index

The CIE General Color Rendering Index $R_a$ quantifies how faithfully a test illuminant renders colors compared to a reference illuminant (a blackbody at the same CCT). For each of 14 test color samples (TCS) with reflectance $\rho_i(\lambda)$, the reflected spectrum under the test illuminant is

$$S_i^{\mathrm{test}}(\lambda) = \hat{S}(\lambda) \cdot \rho_i(\lambda) \tag{7}$$

where $\hat{S}$ is the Y-normalized test spectrum ($Y = 100$). The reflected spectrum is converted to CIE 1964 $U^*V^*W^*$ coordinates with von Kries chromatic adaptation:

$$W^* = 25\,Y^{1/3} - 17 \tag{8}$$

$$U^* = 13\,W^* \cdot (u' - u_0) \tag{9}$$

$$V^* = 13\,W^* \cdot (v' - v_0) \tag{10}$$

where $(u_0, v_0)$ is the adapted white point. The color difference between test and reference renderings is

$$\Delta E_i = \sqrt{(\Delta U_i^*)^2 + (\Delta V_i^*)^2 + (\Delta W_i^*)^2} \tag{11}$$

and the special color rendering index for sample $i$ is

$$R_i = 100 - 4.6\,\Delta E_i. \tag{12}$$

The general index averages the first eight (unsaturated pastel) samples:

$$R_a = \frac{1}{8}\sum_{i=1}^{8} R_i. \tag{13}$$

Notably, $R_9$ (saturated red) through $R_{14}$ are *not* included in $R_a$. This is the well-known "CRI loophole": a spectrum can score $R_a \geq 80$ while rendering saturated reds poorly ($R_9 < 0$), which is precisely the scenario explored by the inverse problem mode of `photopic`.

### 2.6 The optimization problem

In the forward mode (`-run`), `photopic` solves:

$$\boxed{\max_{\mathbf{p}}\ \eta\big(S(\lambda;\mathbf{p})\big) \quad \text{subject to} \quad \begin{cases} R_a \geq R_a^{\min} \\ \min_{i\in 1..8} R_i \geq R_a^{\min} - 10 \\ R_9 \geq R_9^{\min} \\ |\mathrm{CCT} - T_{\mathrm{target}}| \leq 50\,\mathrm{K} \\ D_{uv} \leq 0.012 \end{cases}} \tag{14}$$

In the inverse mode (`-run-maxr9`):

$$\boxed{\max_{\mathbf{p}}\ R_9\big(S(\lambda;\mathbf{p})\big) \quad \text{subject to} \quad \begin{cases} R_a \geq R_a^{\min} \\ \min_{i\in 1..8} R_i \geq R_a^{\min} - 10 \\ \eta \geq \eta_{\min} \\ |\mathrm{CCT} - T_{\mathrm{target}}| \leq 50\,\mathrm{K} \\ D_{uv} \leq 0.012 \end{cases}} \tag{15}$$

Both are solved by COBYLA, Powell's derivative-free constrained optimizer [2], which constructs linear approximations to the objective and constraint functions within a shrinking trust region.

## 2.7 Philips phosphor-converted LED model

Real white LEDs produce light by pumping a short-wavelength (blue) LED and converting part of the output to longer wavelengths via phosphor down-conversion. This process incurs two losses:

1. **Stokes loss.** A photon at wavelength $\lambda > \lambda_{\text{pump}}$ carries less energy than the pump photon that created it. The energy ratio is $\lambda/\lambda_{\text{pump}} > 1$.
2. **Quantum efficiency.** Not every pump photon produces an output photon; the average quantum efficiency is $\eta_{\text{QE}} < 1$.

Following the model used by Philips Research, we define the *cost function*:

$$\boxed{C(\lambda) = \max\left(1,\ \frac{\lambda}{\lambda_{\text{pump}} \cdot \eta_{\text{QE}}}\right)} \tag{16}$$

where $\lambda_{\text{pump}} = 450\,\text{nm}$ is the blue pump wavelength and $\eta_{\text{QE}} = 0.90$ is the average phosphor quantum efficiency. $C(\lambda)$ gives the watts of electrical (pump) power needed per watt of optical output at wavelength $\lambda$.

The *Philips-corrected luminous efficacy* replaces the simple power integral in the denominator of (2) with the cost-weighted integral:

$$\boxed{\eta_{\text{P}} = K_m \frac{\displaystyle\int_{\lambda_0}^{\lambda_1} S(\lambda)\,\bar{y}(\lambda)\,d\lambda}{\displaystyle\int_{\lambda_0}^{\lambda_1} S(\lambda)\,C(\lambda)\,d\lambda}} \tag{17}$$

This penalizes spectra with strong red content, since red photons at $\lambda \approx 650\,\text{nm}$ cost $C(650) = 650/(450 \times 0.90) \approx 1.60$ times as much pump power as blue photons.

The Philips-corrected inverse mode (`-run-maxr9-philips`) solves:

$$\boxed{\max_{\mathbf{p}}\ R_9\big(S(\lambda;\mathbf{p})\big) \quad \text{subject to} \quad \begin{cases} R_a \geq R_a^{\min} \\ \min_{i\in 1..8} R_i \geq R_a^{\min} - 10 \\ \eta_{\text{P}} \geq \eta_{\min} \\ |\text{CCT} - T_{\text{target}}| \leq 50\,\text{K} \\ D_{uv} \leq 0.012 \end{cases}} \tag{18}$$

Note that $\eta_{\text{P}}$ is computed fresh at each optimizer iteration from the current spectrum $S(\lambda;\mathbf{p})$, so the Philips loss model is fully integrated into the optimization loop, not applied as a post-hoc correction.

## 2.8 Parametric spectrum

The spectrum is parameterized as

$$S(\lambda;\mathbf{p}) = B(\lambda,T) \cdot \exp\left(\sum_j p_j\,\phi_j(\lambda)\right) \tag{19}$$

where $\phi_j(\lambda)$ are hat functions (piecewise-linear basis functions) centered at uniformly spaced nodes $\lambda_j = \lambda_0 + j \cdot \Delta\lambda$ with $\Delta\lambda = (\lambda_1 - \lambda_0)/(n-1)$. The exponent $f(\lambda) = \sum_j p_j \phi_j(\lambda)$ is the linear interpolant of the parameter values $p_j$ at the nodes. At $\mathbf{p} = \mathbf{0}$, $S = B$: the unperturbed blackbody. The exponential ensures $S(\lambda) > 0$ for all $\mathbf{p}$.

The hierarchical subdivision doubles the number of nodes at each stage, interpolating the coarse solution onto the fine grid:

$$n \in \{2,\ 3,\ 5,\ 9,\ 17,\ 33,\ 65,\ 129\} \tag{20}$$

$$\Delta\lambda \in \{390,\ 195,\ 97.5,\ \ldots,\ 3.05\}\ \text{nm}. \tag{21}$$

## References


[1] M. Nyström, *Theoretical Limits of Luminous Efficacy under Color Rendering Constraints: A Computational Study*, Technical Report, 2026. Available at https://github.com/IntelLabs/async-toolkit, branch `claude_edits`.

[2] M. J. D. Powell, "A direct search optimization method that models the objective and constraint functions by linear interpolation," in *Advances in Optimization and Numerical Analysis*, S. Gomez and J.-P. Hennart, eds., pp. 51–67, Kluwer, 1994.

[3] W. A. Thornton, "Luminosity and color-rendering capability of white light," *J. Opt. Soc. Am.*, vol. 61, no. 9, pp. 1155–1163, 1971.

[4] W. Walter, "An optimization approach to the computation of luminous efficacy bounds for sources with color rendering constraints," *Color Res. Appl.*, vol. 3, no. 2, pp. 83–90, 1978.


# 3 Building

## 3.1 Prerequisites

- A working CM3 (Critical Mass Modula-3) compiler, installed and in your `PATH` (or referenced by absolute path).
- The `m3utils` tree built—`photopic` depends on several libraries within the tree including `mscheme`, `modula3scheme`, `parseparams`, `integrate`, `cobyla`, `minimize`, `mpfr`, and `cit_util`.
- The `sstubgen` tool must also be available, since the build generates Scheme stubs for the Modula-3 interfaces.

## 3.2 Build procedure

Build the entire `m3utils` tree first (if not already done), then build `photopic`:

```
# Build the full m3utils tree (from the m3utils root)
cd m3utils
export CM3=yes
gmake

# Or build just photopic (after dependencies are built)
cd photopic/src
cm3 -override
```

The binary is placed in a target-specific directory alongside the source, e.g., `photopic/ARM64_DARWIN/photopic` on Apple Silicon macOS or `photopic/AMD64_LINUX/photopic` on x86-64 Linux.

If `sstubgen` is not in your default `PATH`, prefix the build command:

```
env PATH="/path/to/cm3/install/bin:${PATH}" cm3 -override
```

# 4 Command-Line Interface

`photopic` is a Modula-3 program that embeds MScheme, a Scheme interpreter. The main optimization logic lives in `photopic.scm`, which is loaded at startup via the `-scmfile` flag. All command-line invocations require the Scheme mode flags:

```
photopic <mode> <args...> -scm -scmfile photopic.scm
```

The `-scm` flag activates the embedded Scheme interpreter. The `-scmfile` flag specifies a Scheme source file to load (the path to `photopic.scm` relative to the working directory or as an absolute path).

## 4.1 `-run`: Maximize efficacy

```
photopic -run <cct> <min-cri> <min-r9> -scm -scmfile photopic.scm
```

Maximizes luminous efficacy of radiation (LER, in lm/W) subject to:

- CRI (Ra) $\geq$ `<min-cri>`
- $R_9 \geq$ `<min-r9>`
- Worst individual $R_i$ ($i \in 1..8$) $\geq$ `<min-cri>`$-10$
- CCT within $\pm 50\,\mathrm{K}$ of `<cct>`
- $D_{uv} \leq 0.012$

The optimizer runs 8 hierarchical levels (2 to 129 parameters). Set `<min-r9>` to `-100` to leave $R_9$ effectively unconstrained.

## 4.2 `-run-maxr9`: Maximize $R_9$ (inverse problem)

```
photopic -run-maxr9 <cct> <min-cri> <min-efficacy> \
         -scm -scmfile photopic.scm
```

Maximizes the $R_9$ color rendering index subject to:

- CRI (Ra) $\geq$ `<min-cri>`
- Worst individual $R_i \geq$ `<min-cri>`$-10$
- Efficacy $\geq$ `<min-efficacy>` lm/W
- CCT within $\pm 50\,\mathrm{K}$ of `<cct>`
- $D_{uv} \leq 0.012$

This is the inverse of `-run`: instead of finding how efficient a spectrum can be, it finds how well saturated reds can be rendered at a given efficiency floor.

### 4.3 `-run-maxr9-philips`: Maximize $R_9$ with Philips correction

```
photopic -run-maxr9-philips <cct> <min-cri> <min-efficacy> \
         -scm -scmfile photopic.scm
```

Same as `-run-maxr9` but uses the Philips phosphor-converted LED loss model (Section 2.7) in the efficacy constraint. The efficacy floor is interpreted as Philips-corrected lm/W, which accounts for Stokes shift and quantum efficiency losses. Typical Philips-corrected values are 30–40% lower than uncorrected LER.

Output files use the prefix `maxR9P` instead of `maxR9`.

### 4.4 `-run-drop`: Multi-run with TCS axis dropping

```
photopic -run-drop <cct> <min-cri> -scm -scmfile photopic.scm
```

Performs multiple rounds of efficacy maximization. After each round, the test color sample axis with the worst $R_i$ is removed from the CRI average, and the optimizer re-runs against the remaining axes. This explores how much additional efficacy can be gained by sacrificing individual color rendering dimensions.

## 5 Worked Examples

All examples assume you are in the `report-runs/` directory with the `photopic` binary at `../ARM64_DARWIN/photopic` (adjust the path for your platform) and `photopic.scm` at `../src/photopic.scm`.

For brevity, we define:

```
PHOTOPIC=../ARM64_DARWIN/photopic
SCMFILE=../src/photopic.scm
```

### 5.1 Example 1: Basic run at CRI $\geq$ 80

```
$PHOTOPIC -run 2700 80 -100 -scm -scmfile $SCMFILE
```

This maximizes efficacy for a 2700 K source with CRI $\geq$ 80 and no $R_9$ constraint. Expect the optimizer to produce a spectrum with efficacy around 400–420 lm/W, achieved by concentrating power in spectral regions where the eye is most sensitive while maintaining adequate color rendering. Output files are written to the current directory (see Section 6).

### 5.2 Example 2: High CRI at CRI $\geq$ 95

```
$PHOTOPIC -run 2700 95 -100 -scm -scmfile $SCMFILE
```

With a tighter CRI constraint, the optimizer has less freedom to reshape the spectrum. Expect efficacy around 350–370 lm/W—a significant penalty compared to CRI $\geq$ 80, because the spectrum must remain closer to a blackbody shape to render all 8 primary test colors faithfully.

### 5.3 Example 3: Moderate CRI with $R_9$ constraint

```
$PHOTOPIC -run 2700 82 50 -scm -scmfile $SCMFILE
```

This adds an $R_9 \geq 50$ constraint, requiring decent saturated red rendering. Without this constraint, the optimizer often sacrifices deep red content (which the eye is insensitive to) for higher efficacy. The $R_9$ floor forces retention of spectral power near 630 nm, reducing achievable efficacy.

### 5.4 Example 4: Inverse problem—maximize $R_9$ at efficacy $\geq$ 350

```
$PHOTOPIC -run-maxr9 2700 80 350 -scm -scmfile $SCMFILE
```

Asks: given a 2700 K source that must achieve CRI $\geq$ 80 and efficacy $\geq$ 350 lm/W, what is the best $R_9$ possible? This traces the Pareto frontier between efficacy and red rendering quality.

### 5.5 Example 5: Running the full CRI sweep

The script `run-optimizations.sh` automates running at multiple CRI levels:

```
./run-optimizations.sh            # all levels: 60 70 80 82 85 90 95 98
./run-optimizations.sh 90         # only CRI >= 90
./run-optimizations.sh 80 85 90   # specific levels
```

### 5.6 Example 6: Sweeping the efficacy–$R_9$ frontier

The script `run-maxr9-sweep-parallel.sh` sweeps Philips-corrected efficacy from 200 to 320 lm/W, maximizing $R_9$ at each level using 10 parallel processes:

```
./run-maxr9-sweep-parallel.sh          # CRI >= 80 (default)
./run-maxr9-sweep-parallel.sh 95       # CRI >= 95
```

The sweep uses the Philips phosphor-converted LED loss model (Section 2.7) so that the efficacy constraint reflects real-world LED physics. Each run produces `maxR9P` result files.

### 5.7 Example 7: Single Philips-corrected run

```
$PHOTOPIC -run-maxr9-philips 2700 80 270 -scm -scmfile $SCMFILE
```

Maximizes $R_9$ at CRI $\geq$ 80 with Philips-corrected efficacy $\geq$ 270 lm/W. This is equivalent to roughly 400 lm/W uncorrected LER. The Philips model penalizes red spectral content since long-wavelength photons require more pump energy to produce via phosphor conversion.

### 5.8 Example 8: Plotting results

After running optimizations, generate publication-quality figures:

```
python3 plot_spectra.py                # forward problem spectra
python3 plot_maxr9_sweep.py --philips  # Philips-corrected R9 frontier
python3 plot_maxr9_sweep.py 95 --philips  # CRI >= 95
```

This produces PDF and PNG files for:

- Blackbody vs. optimized spectrum comparison

- Multi-panel CRI comparison
- Convergence across resolution levels
- Overlay of all CRI-constrained spectra
- Efficacy vs. CRI trade-off curve
- $R_9$ vs. efficacy Pareto frontier (uncorrected and Philips-corrected)
- Individual spectrum plots at each efficacy level

# 6 Output Files

All output is written to the current working directory.

## 6.1 File naming convention

Output files follow the pattern:

`<cct>_CRI<min-cri>_<suffix>_<dims>.<ext>`

where:

- `<cct>` is the target CCT (e.g., `2700`)
- `<min-cri>` is the minimum CRI constraint (e.g., `82`)
- `<suffix>` encodes run parameters (`R9=-100` for unconstrained $R_9$; `_maxR9_eff350` for the inverse problem)
- `<dims>` is the parameter count at the subdivision level (2, 3, 5, 9, 17, 33, 65, or 129)

One file is produced *at each subdivision level*, so a complete run at 8 levels generates 8 `.res` and 8 `.dat` files.

## 6.2 `.res` files: Optimization results

Each `.res` file is an S-expression on two lines.

**Line 1: Computed specifications.** A flat list with the following fields:

| Position | Field | Description |
|---|---|---|
| 1 | CRI (Ra) | Average of $R_1 \dots R_8$ |
| 2 | Worst $R_i$ | Minimum of $R_1 \dots R_8$ |
| 3 | Efficacy | Luminous efficacy of radiation (lm/W) |
| 4 | CCT | Correlated color temperature (K) |
| 5 | $D_{uv}$ | Distance from Planckian locus |
| 6–19 | $R_1 \dots R_{14}$ | Individual color rendering indices |
| 20 | CRI-14 | Average of all 14 $R_i$ values |
| 21 | Worst-14 | Minimum of all 14 $R_i$ values |
| 22 | Sel-CRI | CRI over selected TCS subset |
| 23 | Sel-worst | Worst $R_i$ over selected subset |
| 24 | Worst axis | Index of the worst TCS axis |

Example (from `2700_CRI82_R9=-100_129.res`):

```
(83.22 72.01 408.61 (2650.00 0.012) (93.16 87.51
 72.01 92.19 80.80 75.68 92.36 72.02 28.67 57.20
 83.54 16.30 90.87 80.46) 73.06 16.30 73.06 16.30 12)
```

This result achieves 408.6 lm/W at CRI = 83.2 with CCT = 2650 K and $R_9 = 28.7$.

**Line 2: Parameter vector.** The optimized parameter values for the piecewise-linear log-modifier. These can be used to reconstruct the spectrum or as a warm-start for further optimization.

### 6.3 `.dat` files: Spectral data

Two types of `.dat` files are produced:

- `w_<name>_<dims>.dat` — the Y-normalized optimized spectrum
- `base_<cct>.dat` — the unmodified blackbody reference at temperature `<cct>`

Both are two-column text files (wavelength in meters, spectral power density), with 391 data points at 1 nm resolution spanning 380–770 nm:

```
3.8e-7 5.366289953705953e6
3.8195e-7 5.7351389943960775e6
3.839e-7 6.1250440250444e6
...
```

To convert wavelengths to nanometers, multiply the first column by $10^9$.

## 7 Architecture

### 7.1 Parametric spectrum model

The optimized spectrum is defined as:

$$S(\lambda) = B(\lambda, T) \cdot \exp\big(f(\lambda)\big)$$

where $B(\lambda, T)$ is the Planck blackbody spectrum at the target CCT, and $f(\lambda)$ is a piecewise-linear function defined by a parameter vector $\mathbf{p} = (p_0, p_1, \ldots, p_{n-1})$.

The parameter nodes are uniformly spaced across 380–770 nm. Between nodes, $f$ is linearly interpolated. At $\mathbf{p} = \mathbf{0}$, the spectrum equals the unmodified blackbody. The exponential ensures the spectrum remains strictly positive for all parameter values.

This representation is implemented in `ParametricSpectrum.m3`. The function `GetFunc` returns an `LRFunction.T` that evaluates $S(\lambda)$ given a parameter vector; `EvalFunction` performs the interpolation lookup and exponential multiplication.

### 7.2 Hierarchical subdivision

Optimization begins at 2 parameters and proceeds through the sequence:

$$2 \to 3 \to 5 \to 9 \to 17 \to 33 \to 65 \to 129$$

At each level, the number of parameters follows $n_{k+1} = 2n_k - 1$ (inserting a midpoint between every adjacent pair). The `Subdivide` procedure interpolates the old parameter vector into the new, higher-resolution grid. The COBYLA initial step size (`rhobeg`) is halved at each level.

This hierarchical approach is essential: starting directly at 129 parameters would produce poor results because COBYLA would get trapped in a local minimum near the blackbody starting point. The low-dimensional stages identify the dominant spectral features (which wavelength regions to emphasize or suppress), and the subsequent stages refine the details.

At 129 parameters, the node spacing is $(770 - 380)/128 \approx 3.05\,\text{nm}$, providing spectral features as narrow as approximately 6 nm (two nodes per feature).

### 7.3 COBYLA optimizer

COBYLA (Constrained Optimization BY Linear Approximation) is a derivative-free optimizer by M. J. D. Powell. It is well suited to this problem because:

- The CRI computation involves color adaptation and nonlinear coordinate transforms that are not easily differentiable.
- The constraint functions are nonlinear and involve search (CCT is found by minimizing distance in MacAdam $uv$ space).
- The number of function evaluations per optimization step scales linearly with the number of parameters.

The Modula-3 binding (`COBYLA_M3.Minimize`) accepts the state vector, number of constraints, an objective/constraint evaluation function, `rhobeg` (initial trust-region radius), `rhoend` (convergence tolerance), and a maximum iteration count.

### 7.4 CRI calculation

Color rendering is evaluated following CIE 13.3:

1. Normalize the test spectrum so $Y = 100$.
2. Determine the CCT by minimizing distance in MacAdam 1960 $uv$ space to the Planckian locus (Brent's method via `search-T`).
3. Generate the reference illuminant: a blackbody at the CCT.
4. For each of 14 test color samples (TCS), compute the reflected spectrum under both the test and reference illuminants.
5. Convert to CIE 1964 $U^*V^*W^*$ coordinates with von Kries chromatic adaptation.
6. $R_i = 100 - 4.6\,\Delta E_i$ where $\Delta E_i$ is the Euclidean distance between adapted and reference $U^*V^*W^*$ values.
7. CRI (Ra) $= \frac{1}{8}\sum_{i=1}^{8} R_i$.

The test color sample reflectance spectra are tabulated at 1 nm resolution in `Tcs.i3` (14 samples, covering pastels $R_1$–$R_8$ and saturated colors $R_9$–$R_{14}$). CIE $\bar{x}$, $\bar{y}$, $\bar{z}$ color matching functions are tabulated in `CieXyz.i3`; the photopic luminous efficiency function (683 lm/W peak) is in `CieSpectrum.i3`.

### 7.5 Constraint convention

In the objective/constraint function returned to COBYLA, the first element of the returned vector is the quantity to *minimize* (e.g., negative efficacy to maximize efficacy, or negative $R_9$ to maximize $R_9$). Subsequent elements are constraint values where **positive means satisfied**:

$$c_i \geq 0 \quad \Longleftrightarrow \quad \text{constraint } i \text{ is met}$$

For example, the CRI constraint returns CRI − min_cri, and the CCT constraint returns two values: CCT − (target − 50) and (target + 50) − CCT.

## 8 Interactive Use

When invoked with `-scm -scmfile photopic.scm` but *without* a `-run`, `-run-maxr9`, or `-run-drop` flag, `photopic` drops into an interactive MScheme read-eval-print loop (REPL). This is useful for exploration, debugging, and prototyping new objective functions.

```
$PHOTOPIC -scm -scmfile ../src/photopic.scm
```

### 8.1 Useful interactive commands

**Setting up a problem**

```
;; Create a parametric spectrum problem at 3000K with 5 parameters
(setup-problem!
  (ParametricSpectrum.MakeBlackbodyInWavelength 3000 l0 l1)
  5)
```

**Inspecting the current spectrum**

```
;; Compute full specs of the current spectrum
(calc-specs w)
;; => (cri-ra worst-ri efficacy (cct duv) (R1..R14) ...)

;; Get just the efficacy
(total-lumens-per-watt w)
```

**Running one optimization pass**

```
;; Run COBYLA with rhobeg=2 using the efficacy target
(run 2 specs->target)

;; Subdivide and run again at finer resolution
(subdivide-problem!)
(run 1 specs->target)
```

**Evaluating a specific spectrum**

```
;; CRI of a 4000K blackbody
(calc-cri (make-Bl 4000))

;; Efficacy of a fluorescent illuminant
(total-lumens-per-watt (FL 4))
```

**Plotting**

```
;; Write the current optimized spectrum to a gnuplot-compatible file
(plot (normalize-spectrum w) l0 l1 "my_spectrum.dat" 391)

;; Write the 14 TCS reflectance spectra
(plot-the-samples)
```

**Modifying optimization parameters**

```
;; Change target CCT
(set! *target-cct* 4000)

;; Change minimum CRI
(set! *min-cri-ra* 90)

;; Change minimum R9
(set! *min-r9* 50)

;; Change minimum efficacy (for -run-maxr9 mode)
(set! *min-efficacy* 350)
```

**Reloading after edits**

```
;; Reload photopic.scm after editing it
(reload)
```

# 9 Customization

## 9.1 Writing new objective functions

The optimizer calls a *spec-evaluator* function that takes the output of `calc-specs` and returns a vector for COBYLA. The existing evaluators follow this pattern:

```
(define (specs->my-target specs)
  (let ((lpm    (caddr specs))        ;; efficacy (lm/W)
        (r9     (nth (car (cddddr specs)) 8))  ;; R9
        (cri    (car specs))          ;; CRI (Ra)
        (crmin  (cadr specs))         ;; worst Ri (i in 1..8)
        (cct    (caar (cdddr specs))) ;; CCT (K)
        (Duv    (cadar (cdddr specs)));; Duv
        )
    (list <objective>        ;; minimize this (negate to maximize)
          <constraint-1>     ;; positive = satisfied
          <constraint-2>
          ...
          )))
```

The return value is a list whose first element is the objective to minimize and whose remaining elements are constraint values (positive means feasible). Update `*num-constraints*` if you change the number of constraints from the default 6.

## 9.2 What `calc-specs` returns

The `calc-specs` function evaluates a spectrum and returns a list with the following structure:

```
(cri-ra              ;; (car specs)      -- CRI average R1..R8
 worst-ri            ;; (cadr specs)     -- min of R1..R8
 efficacy            ;; (caddr specs)    -- lm/W
 ((cct duv)          ;; (cadddr specs)   -- CCT and Duv
  (R1 R2 ... R14))   ;; individual Ri values
 cri-14              ;; average of all 14 Ri
 worst-14            ;; min of all 14 Ri
 sel-cri             ;; CRI over selected subset
 sel-worst           ;; worst over selected subset
 worst-axis)         ;; index of worst TCS axis
```

## 9.3 Accessing individual fields

```
;; Given: (define s (calc-specs w))
(car s)                          ;; CRI (Ra)
(cadr s)                         ;; worst Ri
(caddr s)                        ;; efficacy
(caar (cdddr s))                 ;; CCT
(cadar (cdddr s))                ;; Duv
(nth (car (cddddr s)) 8)         ;; R9 (9th of 14, 0-indexed)
(nth (car (cddddr s)) 0)         ;; R1
```

## 9.4 Example: maximize efficacy subject to $R_9 \geq 50$

```
(define (specs->eff-r9floor specs)
  (let ((lpm  (caddr specs))
        (r9   (nth (car (cddddr specs)) 8))
        (cri  (car specs))
        (cct  (caar (cdddr specs)))
        (Duv  (cadar (cdddr specs))))
    (list (- lpm)                        ;; maximize efficacy
          (- cri 80)                     ;; CRI >= 80
          (- r9 50)                      ;; R9  >= 50
          (- cct (- *target-cct* 50))    ;; CCT in range
          (- (+ *target-cct* 50) cct)
          (* 1000 (- 0.012 Duv)))))      ;; Duv limit

;; Run it:
(set! *num-constraints* 5)
(run-example-iters! 2700 80 50 7 specs->eff-r9floor "_custom")
```

# 10 Reference

## 10.1 Global variables

| Variable | Default | Purpose |
|---|---|---|
| `*target-cct*` | 2700 | Target correlated color temperature (K) |
| `*min-cri-ra*` | 82 | Minimum CRI (Ra) constraint |
| `*min-r9*` | $-100$ | Minimum $R_9$ constraint |
| `*min-efficacy*` | 300 | Minimum efficacy (lm/W, for `-run-maxr9`) |
| `*pump-nm*` | $450 \times 10^{-9}$ | Blue LED pump wavelength (m) |
| `*pump-qe*` | 0.90 | Phosphor quantum efficiency |
| `*num-constraints*` | 6 | Number of COBYLA constraints |
| `l0` | $380 \times 10^{-9}$ | Blue limit of visible spectrum (m) |
| `l1` | $770 \times 10^{-9}$ | Red limit of visible spectrum (m) |

## 10.2 Key source files

| File | Contents |
|---|---|
| `src/photopic.scm` | Main optimization logic (Scheme) |
| `src/Main.m3` | Program entry point; Scheme REPL setup |
| `src/ParametricSpectrum.i3` | Parametric spectrum interface |
| `src/ParametricSpectrum.m3` | Parametric spectrum implementation |
| `src/CieSpectrum.i3` | Photopic luminous efficiency data |
| `src/CieXyz.i3` | CIE $\bar{x}\bar{y}\bar{z}$ color matching functions |
| `src/Tcs.i3` | Test color sample reflectance spectra (1–14) |
| `src/TempUv.m3` | Temperature-to-$uv$ lookup table |
| `src/Blackbody.m3` | Planck blackbody radiance |
| `src/m3makefile` | Build dependencies and Scheme stub generation |

## 10.3 Physical constants

| Symbol | Value | Defined as |
|---|---|---|
| $c$ | 299 792 458 m/s | Speed of light |
| $h$ | $6.626\,070\,15 \times 10^{-34}$ J s | Planck's constant |
| $k$ | $1.380\,649 \times 10^{-23}$ J/K | Boltzmann's constant |